0703. Kth Largest Element in a Stream

ory: Heap / Priority Queue | LeetCode: https://leetcode.com/problems/kth-largest

Problem

- Design a class that, after each add(val), returns the k-th largest element seen so far.

Intuition

- Keep only the k largest values in a min-heap of size k. Its top is the smallest of the top k, which is the k-th largest overall.

Brute Force

- Keep a sorted list and insert each value: O(n) per add.

Optimal Approach

- Constructor: add every initial number through add().
- Add(val): offer val; if size > k, poll the minimum. Return peek().

Worked Example

```
k = 3, nums = [4,5,8,2].

init heap keeps {4,5,8}
add 3  -> {3,4,5,8} -> drop 3 -> top 4
add 5  -> {4,5,5,8} -> drop 4 -> top 5
add 10 -> drop 5 -> {5,8,10} top 5
add 9  -> drop 5 -> {8,9,10} top 8
add 4  -> drop 4 -> top 8
```

Java Solution

```java
public class KthLargest {
    private final PriorityQueue<Integer> heap = new PriorityQueue<>();
    private final int k;

    public KthLargest(int k, int[] nums) {
        this.k = k;
        for (int x : nums) add(x);
    }

    public int add(int val) {
        heap.offer(val);
        if (heap.size() > k) heap.poll();
        return heap.peek();
    }
}
```

Complexity

- Add: O(log k). Space: O(k).

Pitfalls

- Min-heap, not max-heap: we discard the smallest of the candidates.